Docker and renv strategy

R.G. Thomas

Contents

1 Introduction

Reproducibility is a cornerstone of professional data analysis, yet achieving it with R Markdown can be challenging. R projects often break on other computers due to mismatched R versions or package versions, sending developers into "dependency hell." To solve this, we can leverage **renv** (for R package management) and **Docker** (for containerizing the computing environment). Together, these tools ensure that an R Markdown document runs anywhere with the same packages, R version, and system libraries as the original setup. Below, we present a structured guide on using renv and Docker for a reproducible R Markdown workflow, with diagrams, code snippets, and clear steps.

R is an open-source programming language for statistics and data science. The **renv** package helps create reproducible R project environments by isolating package dependencies.

1.1 Using renv for R Package Reproducibility

renv (Reproducible Environment) is an R package that manages library dependencies for your project. Instead of sharing one system library across projects, renv gives each project its own library of R packages and a lockfile (`renv.lock`) recording exact package versions. This means that if two developers use renv, they can install identical package versions for a project, avoiding version conflicts and "it works on my machine" issues. In practice, renv makes it easy to **snapshot** your R packages and later **restore** them on any system.

1.1.1 Key Features of renv:

- **Isolated project library**: renv creates a project-specific library (usually in `renv/library`) containing all packages used by that project. This isolation means updates in one project won't affect others.
- **Lockfile for dependencies**: When you finish installing or updating packages, you call `renv::snapshot()`. This produces `renv.lock`, a JSON file listing each package and the exact version (and source) in use. The lockfile is meant to be committed to version control and shared.
- **Restoring environment**: On a new machine (or when reproducing past results), you use `renv::restore()` to install the **exact versions** of packages from the lockfile. This gives an R environment **identical** to the one that created the lockfile, as long as the same R version is available.

1.1.2 Example: Using renv in an R Project

```r
# Install and initialize renv in a new R project
install.packages("renv")      # one-time installation of renv
renv::init()                  # initialize renv (creates renv infrastructure)

# ...After installing or updating some project-specific packages...
renv::snapshot()              # save exact package versions to renv.lock

# ...Later or on another system...
renv::restore()               # restore packages from renv.lock for reproducibility
```

In an R Markdown workflow, you would: attach renv to the project, work on your `.Rmd` using whatever CRAN/Bioconductor packages needed, and finally snapshot the environment. The `renv.lock` file ensures that anyone with the lockfile can recreate the **same R package environment**. However, renv alone does **not lock the R version or system libraries** – that's where Docker comes in.

1.2 Using Docker for OS-Level Reproducibility

While renv handles R packages, **Docker** takes care of the **operating system, R interpreter, and system libraries**. A Docker container is like a lightweight virtual machine image that includes everything needed to run your project: the specific OS (Linux distribution), exact R version, required system libraries (e.g. C/C++ libraries), and your R code. By running an R Markdown project in Docker, you ensure that differences in OS or R installation are no longer an issue – any machine running Docker will run the container *identically*.

Docker allows you to **build an image** that encapsulates your environment. For R workflows, a common practice is to start from

a base image (for example, the Rocker project provides images like `rocker/r-ver:<R-version>` for R). You then add your content and dependencies.

1.2.1 Example: A Simple Dockerfile for an R Markdown Project

```
# Use R 4.1.0 on Linux as base image
FROM rocker/r-ver:4.1.0

# Copy renv lockfile into the image
COPY renv.lock /renv.lock

# Install renv and restore packages from lockfile
RUN R -e "install.packages('renv', repos='https://cloud.r-project.org'); renv::restore()"

# Copy the rest of the project (e.g., R Markdown files)
COPY . /workspace

# Set working directory and default command (if needed)
WORKDIR /workspace
CMD ["Rscript", "-e", "rmarkdown::render('analysis.Rmd')"]
```

In the snippet above, the Docker image will install the exact package versions from `renv.lock`. The `renv::restore()` command inside `RUN` will install all the R packages (and exact versions) listed in the lockfile into the image's R library.

1.3 Combining renv and Docker in R Markdown Workflows

Using renv or Docker alone improves reproducibility, but **combining them gives the best of both worlds**. Docker will guarantee the OS and R version, and renv will guarantee the R packages. Together, you achieve **end-to-end reproducibility** — from operating system all the way to the R code's package versions.

1.3.1 Steps to Integrate renv with Docker:

1. **Develop with renv**: Start a new R project for your R Markdown analysis and run `renv::init()`. Install all the R packages your `.Rmd` needs. Periodically run `renv::snapshot()` to update the lockfile.
2. **Write a Dockerfile**: Define the base image (e.g., `FROM rocker/r-ver:4.1.0`), copy the `renv.lock`, install dependencies, and set up the project.
3. **Build the Docker image**: Run `docker build -t myanalysis/image:v1 .` to create the image.

4. **Share the image**: Push it to Docker Hub or share the file, so others can pull and run it.
5. **Run anywhere with confidence**: The container ensures consistency across machines without additional manual setup.

1.4 Conclusion

By incorporating **renv** and **Docker**, an R Markdown project becomes **easy to share and fully reproducible**. The original author can provide a Docker image (or Dockerfile) along with the R Markdown and `renv.lock`. A collaborator or reviewer can launch the container and get the **same results**, without worrying about package versions or system setup. This approach is considered a **best practice for long-term reproducibility in R**.

Together, **renv and Docker** ensure that your R Markdown analyses are **completely reproducible and portable**, meeting the high standards required for professional data science and research documentation.